

Containers: A Mini Guide to create containers in Windows laptop

What is needed? A Provider. Example: Docker Desktop

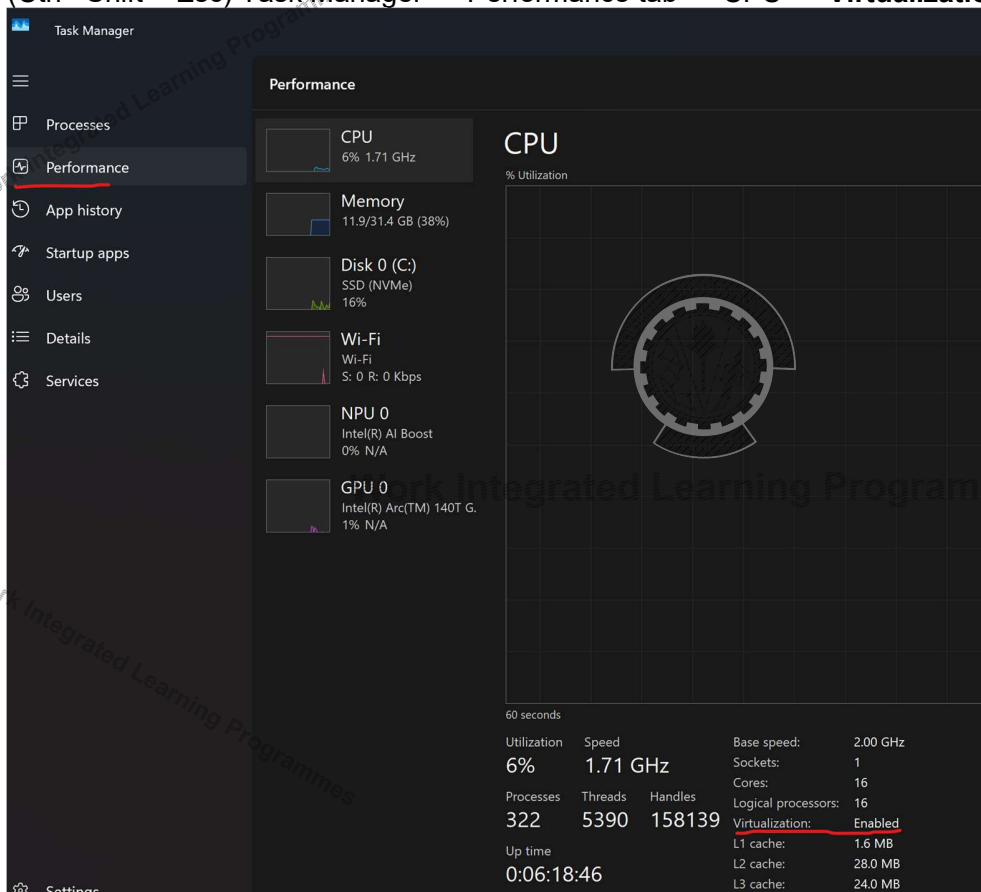
Is Docker Desktop the only provider for Windows: No. There are others: Podman Desktop, Rancher Desktop, Native Windows Server Containers (no Linux support) etc are available

What does Docker Desktop do? Docker Desktop for Windows enables us to manage containers seamlessly. It supports **Linux containers** (default and most common, running via a lightweight utility virtual machine) and **Windows OS containers** (for native .NET Framework or IIS applications).

What is needed for the Docker to get the containers run on Windows? Hardware virtualization must be enabled in the BIOS/UEFI (Unified Extensible Firmware Interface) for Docker to get the containers run on Windows.

How to verify whether your device already has virtualization enabled?

(Ctrl +Shift + Esc) Task Manager → Performance tab → CPU → **Virtualization: Enabled**. (Bottom right)



If it is not enabled, then enable the same with BIOS/UEFI

Toggle **Intel Virtualization Technology (VT-x / VMX)** for Intel CPUs to **Enabled**. If not Intel, search for your specific CPU.

Is enabling the virtualization sufficient? No.

Run the following commands in PowerShell:

- 1) `Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Windows-Subsystem-Linux, VirtualMachinePlatform -All -NoRestart`
- 2) `wsl -update`

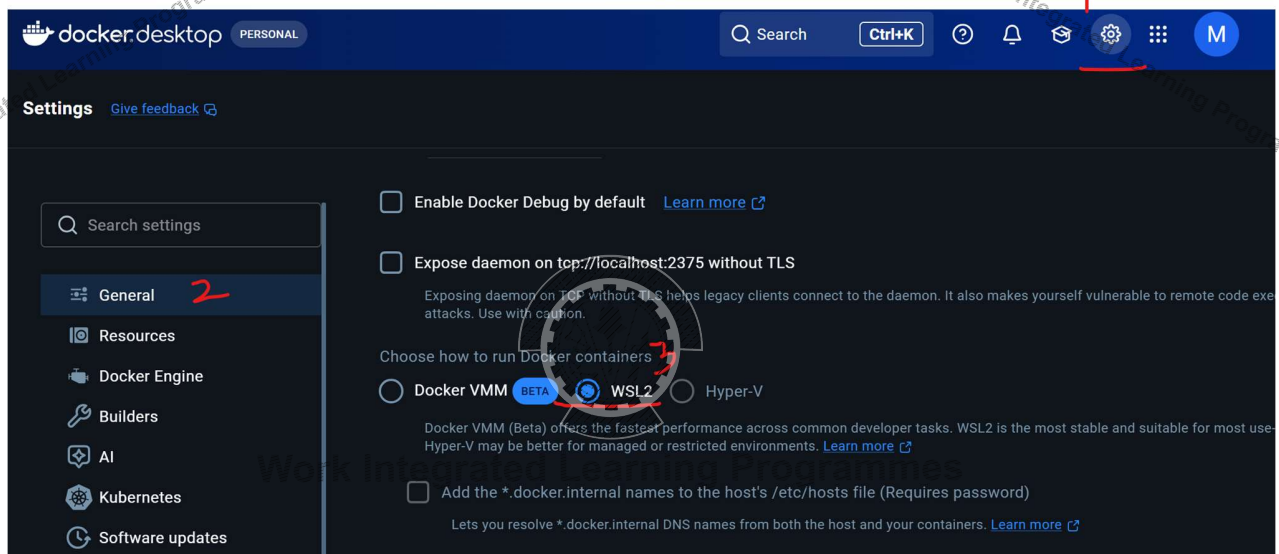
(wsl → Windows Subsystem for Linux)

BIOS activation only unlocks the physical hardware capability. The PowerShell command#1 given above installs the **Windows software layers** required to utilize that hardware:

Microsoft-Windows-Subsystem-Linux (WSL): Enables Windows to run an integrated Linux kernel.
VirtualMachinePlatform: The low-level hypervisor platform that allows Windows to host micro-VMs.

What is the recommended installation for Docker Desktop? Use the Official Website Installer (.exe). The Windows Store layer introduces certain restrictions and hence the official installer is recommended.

Ensure that **WSL2** based engine is checked during the setup wizard. If the setup Wizard is not giving that option (setups keep changing, do check for the options – gear/settings -General -choose how to run docker containers)



After installation, try the following commands:

`docker --version` – to see the version (in a way, whether docker installation was successful)

`docker ps` – to see the running processes. These days, docker desktop by default runs a container (welcome-to-docker)

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
ac8d4ad8abd1	mohanbits/my-python-app:v1.0	"python app.py"	3 hours ago	Up 3 hours	0.0.0.0:5002->5000/tcp, [::]:5002->5000/tcp	web-downloaded-app
a92a7dd18b5b	docker/welcome-to-docker:latest	"/docker-entrypoint..."	6 hours ago	Up 6 hours	0.0.0.0:8088->80/tcp, [::]:8088->80/tcp	welcome-to-docker

A sample output is given above. See the port mapping. 5002 (external) mapped to 5000 (internal).

Can Docker route traffic to multiple containers listening on the same internal port (e.g., Port 80) without conflict? Yes.

How does it achieve? Docker leverages Network Isolation and Port Mapping (Network Address Translation - NAT). **Isolation**: Every container acts as an independent "apartment" with its own private network card and internal IP address (e.g., 172.17.0.2). Container A can use internal port 80, and Container B can use internal port 80 simultaneously because their network boundaries are distinct. **Port Mapping**: Docker translates traffic from the laptop's unique *external* ports to the container's *internal* ports:

Laptop `http://localhost:8080` → Routed to Container A's internal Port 80.

Laptop `http://localhost:8088` → Routed to Container B's internal Port 80.

You will often see the container ip addresses to begin with **172.17.0**.

Container IP addresses start with 172.17.0 because it is the default subnet range assigned by Docker to its default internal bridge network, named **docker0**. When we install and run docker engine, it automatically creates a Linux bridge interface called `docker0`.

Couldn't the docker assign different internal ports automatically to simplify translation? No.

Why? Changing the internal ports introduces complexities given below:

Hardcoded Software defaults: Official software images (like Nginx on port 80 or MySQL on port 3306) expect specific ports. Altering them would require modifying the internal configuration files of third-party apps, breaking Docker's "run anywhere without modification" guarantee.

Standardization: Developers can safely write code assuming every instance of an application defaults to its standard industry port internally.

The important fact: There is not much of a penalty or overhead for port mapping. We can consider it as **Zero-Speed Penalty**. The NAT translation happens at the hardware/kernel level in nanoseconds. So, modifying the internal port yields no speed advantage.

Can an image run identically across Windows, Mac, and Linux?

Yes. Containers package their entire "world". This includes everything starting from the binary runtimes, system tools, and libraries (e.g., Python 3.11 + Flask code). The host system does not need Python installed. The **Docker Engine** serves as a standard abstraction layer. On Linux, it talks directly to the kernel; on Windows, it interacts with WSL 2; on Mac, it uses a hyper-optimized background VM. All systems follow the **Open Container Initiative (OCI)** standard; hence, cross-platform behaviour is guaranteed.

The **Open Container Initiative (OCI)** is an open-source project under the Linux Foundation. to build vendor-neutral, portable, and open standards for software container formats and runtimes. It stops software lock-in by ensuring containers work across different tools and cloud platforms.

Core Specs:

- **Image Specification:** How to build and structure a container image so any system can read it.
- **Runtime Specification:** How to unpack and run the container files on a machine.
- **Distribution Specification:** Sets the standard API rules to safely push and pull images from registries.

How does this help?

- **No Lock-In:** Not forced to use just one specific provider (like Docker)
- **Portability:** An image built in one place runs the same way on any other compliant system.

How does Docker handle CPU architecture differences. For example Intel (x86_64) / Apple (ARM64)?

Software compiled for an Intel CPU cannot natively run on an ARM processor. Docker handles this through Multi-Platform Manifests on Docker Hub. When an image is requested, Docker Hub automatically checks the CPU architecture and serves the corresponding matching slice of code. If an Intel image is forced onto an ARM device, Docker Desktop uses an emulation layer to translate on-the-fly, though it incurs a minor performance overhead.

Usability of Docker Commands (can we use the same command with other providers like Podman?)

Major alternatives include **Podman** and **nerdctl** adhere to the OCI specifications and so most of the commands are functionally identical.

Let's do some minimal exercises:

#1: Spinning up and Inspecting an Nginx Web Server

Launch Container: Run a detached Nginx container mapping external port 8080 to internal port 80.

```
docker run -d -p 8080:80 --name my-web-server nginx
```

Test: `http://localhost:8080`.

Inspect Live Traffic Logs: Stream real-time container access logs (refresh the browser to see updates populate).

```
docker logs -f my-web-server
```

(Ctrl + C → stop tracking logs, but container is not terminated).

See Running Processes:

```
docker ps
```

#2: Modifying Container Content (Windows Native Method)

Note: Instead of the Windows Cmd shell, use PowerShell. PowerShell handles file redirections uniquely.

Create a raw HTML file locally in the current working dir:

```
Set-Content -Path ".\index.html" -Value '<html><body style="font-family:sans-serif; text-align:center; padding-top:100px; background-color:#f4f4f9;"><h1 style="color:#007bff;">Hello from my Windows Laptop as a demo to students!</h1><p>This page is successfully running inside an isolated Docker Container and demoed to students.</p></body></html>'
```

Copy the file directly across the isolation boundary into Nginx's public rendering directory:

```
docker cp .\index.html my-web-server:/usr/share/nginx/html/index.html
```

Refresh `http://localhost:8080` to see the update.

#3: Building a Custom Python Application with a Dockerfile

Initialize Directory:

```
mkdir C:\docker-python-app
```

```
cd C:\docker-python-app
```

Generate Flask Web Script (app.py): (use PowerShell)

```
Set-Content -Path ".\app.py" -Value 'from flask import Flask
app = Flask(__name__)
```

```
@app.route("/")
def home():
    return "<h1>Hello from my Custom Python Container demoed to students!</h1><p>This app was bundled using a Dockerfile.</p>"
```

```
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)'
```

Generate Instructions Layout (Dockerfile):

```
Set-Content -Path ".\Dockerfile" -Value 'FROM python:3.11-slim
WORKDIR /app
RUN pip install flask
COPY app.py .
EXPOSE 5000'
```

```
CMD ["python", "app.py"]'
```

Compile Image Blueprint:

```
docker build -t my-python-app .
```

Run the Compiled Custom Image:

```
docker run -d -p 5001:5000 --name running-python-container my-python-app
```

Verify: <http://localhost:5001>

#4: Distributing Content via Docker Hub

Authenticate Terminal session:

```
docker login -u mohanbits
```

i Info → A Personal Access Token (PAT) can be used instead.
To create a PAT, visit <https://app.docker.com/settings>

Password:

Login Succeeded

Note: You should have signed up with Docker Hub for doing this

Apply User Registration Tag: (Docker tags add names rather than cloning physical files. Images share identical structural asset IDs).

```
docker tag my-python-app mohanbits/my-python-app:v1.0
```

mohanbits → this is the username in the dockerhub

Upload to Global Cloud Storage Registry:

```
docker push mohanbits/my-python-app:v1.0
```

Note: When you push to docker hub, each of your microservice will become a separate repository (recommended). One app/microservice = one project = one repository. We will use different tags to store the various versions.

Repository = project name; Tags = Versions.

Note: as a general practice we never upload multiple microservices in one single repository. A couple of reasons for the same. One of them is that when one of the microservice in a repo is refreshed (a new version tag is available), the CI/CD pipeline gets triggered. Now, the pipeline needs to check which service got updated etc to decide further and take actions (if needed)

Core Concepts

Docker: The underlying platform and engine that allows you to build, run, and share applications in isolated, lightweight environments.

Image: A read-only, executable template containing everything needed to run an application—code, runtime, libraries, and environment variables. Think of it as a class in programming or a blueprint.

Container: A running instance of an image. If an image is the blueprint, the container is the actual house built from it. We can run hundreds of identical containers from a single image simultaneously.

Repository: A collection of **related** Docker images, usually representing different versions of the same application hosted on a registry.

IDs & Tags

Image ID: A unique 64-character alphanumeric string (a SHA-256 hash) that permanently identifies a specific image build.

Container ID: A unique string assigned to a container the moment it is created. Used to target that specific container for lifecycle commands (e.g., docker stop).

Tag: A human-readable label attached to an image, typically used for versioning (e.g., node:18-alpine or ubuntu:latest). If no tag is specified, Docker defaults to latest

Decoding a sample CLI Output

A sample command: docker ps

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
ac8d4ad8abd1	mohanbits/my-python-app:v1.0	"python app.py"	3 hours ago	Up 3 hours	0.0.0.0:5002->5000/tcp, [::]:5002->5000/tcp	web-downloaded-app
a92a7dd18b5b	docker/welcome-to-docker:latest	"/docker-entrypoint..."	6 hours ago	Up 6 hours	0.0.0.0:8088->80/tcp, [::]:8088->80/tcp	welcome-to-docker

Field	What it means?
CONTAINER ID	The unique hash identifying that specific running instance.
IMAGE	The image name and tag used to spawn the container.
COMMAND	The default script or executable the container runs upon booting up.
CREATED	How long ago the container was instantiated
STATUS	The current state and uptime (e.g., Up 3 hours for the python app & 6 hours for the welcome-to-docker container) We can also have others like Exited (0) – not shown here. We need to use docker ps -a for seeing the exited containers. The exit code is given in brackets – 0 is a clean exit (0 is typically considered clean exit for any entity like scripts, a process, return values, last command execution status etc in the Linux world)
PORTS	Network port mappings. An entry for the Python app like 0.0.0.0:5002->5000/tcp means traffic hitting port 5002 on your host machine/laptop is routed to port 5000 inside the container
NAMES	A human-readable name. If not assigned via the --name flag, Docker generates a random name

Extended Concepts/Terminology

Dockerfile: A plain-text document containing the step-by-step instructions (commands) used to assemble and build a new Docker image from scratch.

Volume: A mechanism for persisting data. Containers are ephemeral; if a container is deleted, its internal data is lost. Volumes live outside the container's lifecycle on the host, keeping data safe and persistent.

Network: A virtual bridge that allows containers to communicate securely with each other and the outside world.

Registry: A centralized service that stores and distributes repositories of images. Docker Hub is the default public registry, but private ones are widely used in enterprises.

Docker Manifest: A file or list that describes an image, including information about its layers, size, and the operating system/architecture it supports (e.g., AMD64, ARM64). This is critical for multi-architecture builds, allowing a single tag like python:3.9 to pull the correct binaries depending on the host system's hardware.

Layer: Docker images are constructed from a series of layers. Each instruction in a Dockerfile (like RUN, COPY, ADD) creates a new **immutable** layer. These layers are cached, which dramatically speeds up subsequent builds and saves disk space when multiple images share base layers. •

Docker Daemon (dockerd): The background service running on the host system that does the heavy lifting: building, running, and distributing Docker containers.

The Docker client (CLI) communicates with this daemon via REST APIs.

Build Context: The set of files located in the specified PATH or URL when a docker build command gets executed. The entire context is sent to the Docker Daemon, which is why utilizing a .dockerignore file is a best practice to avoid sending unnecessary gigabytes (like node_modules) to the daemon

Docker Compose

This is a tool for defining and running multi-container applications (like a web app paired with a database) using a single YAML configuration file (docker compose.yml).

Necessity:

Modern apps use a lot of collaborating services: e.g., a web server, a database, and a cache. Starting each one with a separate terminal command is slow and messy.

compose.yml – one single file that has the entire setup. This ensures that the collaborative app runs the exact same way on every computer.

docker compose up—to start the entire application stack; docker compose down—to stop and clean up everything.

Compose automatically creates a private network for the services so they can talk to each other securely without manual setup. We can definitely create the containers separately. In that case, making them communicate requires manual networking configuration.



Work Integrated Learning Programmes

System vs. Application Containers

- Both these types rely on the exact same underlying Linux kernel features (Namespaces and Cgroups).
- Architectural goals are different.

Feature	System Containers (e.g., LXC, LXD)	Application Containers (e.g., Docker, Podman)
Primary Goal	Simulate a full, lightweight virtual machine (VM).	Package and run a single application or service.
Architecture	Runs a full init system (like systemd) that manages multiple background processes (cron, SSH, syslog,)	Runs a single primary process (e.g., a Python app or an Nginx daemon).
State & Lifespan	Persistent and long-lived. They are updated and patched in-place, just like a traditional server.	Ephemeral and stateless. They are never patched; if an update is needed, the container is destroyed and replaced with a new image.
Best Use Case	"Lift-and-shift" migrations of legacy monolithic applications; providing complete sandbox environments for traditional system administration.	Microservices architectures, automated CI/CD pipelines, and cloud-native auto-scaling (e.g., Kubernetes).

An **Application Container** can be considered like a disposable, single-purpose tool. If a web application requires a backend database, we do not install the database inside the web app's container. We deploy two separate application containers and link them (network)

A **System Container** can be considered like a traditional server. We will use a system container when we need the complete environment and behaviour of a Virtual Machine, but with the bare-metal performance and rapid boot times of a container. One Primary Use Case: **Legacy "Lift-and-Shift"**: Migrating older, monolithic applications that require a full OS environment (running multiple daemons, cron jobs, and background logging) without rewriting them for microservices.